Transformer Zero: LoRA Fine-Tuning and From-Scratch Decoding of a Decoder-Only LLM for Criterion-Level Clinical-Trial Eligibility

Wajih-Ur-Raza Asif
Department of Computer Science
FAST–NUCES
Generative AI (Assignment 2)
November 10, 2026

***Abstract*—We fine-tune an open-weight, decoder-only language model to decide, for a single clinical-trial eligibility criterion and a patient note, (i) a five-way eligibility label and (ii) the exact note sentences that justify it. We use the expert-annotated TrialGPT-Criterion-Annotations corpus and a patient-disjoint split so that no patient leaks between training and test. On a held-out test set of 206 patient–criterion pairs, LoRA fine-tuning raises a Qwen3-4B-Instruct model from 0.495 to 0.762 label accuracy and from 0.444 to 0.617 evidence micro-F1, while never emitting malformed output. For context, the dataset's own GPT-4 labels agree with the experts at only 0.903 accuracy, and that number is itself inflated because the gold labels were created by correcting GPT-4, so 0.76 from a 4B model is close to the practical ceiling. Beyond accuracy, we implement the transformer decoding stack from scratch on top of the published Qwen3 weights: a key–value (KV) cache, a tiled online-softmax attention kernel, and greedy speculative decoding with a 0.6B draft model. The KV cache gives up to a 12× decode speed-up at a 1024-token prompt; the tiled kernel reproduces full attention to within floating-point rounding; and speculative decoding, while numerically exact, is a measured *slowdown* at this scale because the small draft is accepted too rarely. A 27-test CPU suite proves our cached, tiled, and speculative paths match Hugging Face token-for-token in float32. All components are exposed through a browser app that screens a patient against a trial criterion-by-criterion with cited evidence.**

***Index Terms*—clinical trial matching, decoder-only transformer, LoRA, KV cache, FlashAttention, speculative decoding, parameter-efficient fine-tuning**

I. Introduction

Recruiting patients is one of the slowest, most expensive steps in running a clinical trial, and much of the cost is the manual labour of reading a patient's record against each eligibility criterion of each candidate trial. The TrialGPT line of work [1] framed this as a criterion-level classification problem: given one eligibility criterion and one patient note, decide whether the patient meets it, and point to the sentences that say so.

This assignment—*Transformer Zero*—asks two things at once. First, a *modelling* question: can a small, open-weight, decoder-only LLM be fine-tuned to produce faithful, machine-checkable criterion-level decisions with evidence? Second, a *systems* question: can the inference machinery that makes such a model usable—the KV cache, the attention kernel, and speculative decoding—be built by hand, understood, and measured, rather than taken on faith from a library? We answer both on the same model and the same data, which lets the report connect decoding behaviour directly to the task it serves.

Contributions.

1) A LoRA fine-tune of Qwen3-4B-Instruct and Qwen3-1.7B that emits strict JSON (label + evidence sentence IDs), trained with the loss masked to the answer tokens only, on a patient-disjoint split (Section IV-A, Section VI).
2) A from-scratch Qwen3 forward pass with a preallocated, rollback-capable KV cache (Section IV-B), benchmarked against recomputation (Section VI-D).
3) A tiled online-softmax attention kernel validated for exactness against a reference implementation and PyTorch SDPA (Section IV-C, Section VI-E).
4) Greedy speculative decoding with a shared-tokenizer draft model, reported *honestly* as a slowdown at this scale with an explained cause (Section IV-D, Section VI-F).
5) An error analysis of all 49 test mistakes (Section VI-C) and a browser screening app (Section IV-F).

II. Related Work

Clinical-trial matching. TrialGPT [1] introduced the end-to-end retrieve/match/rank framework and, crucially for us, released criterion-level expert annotations in which three physicians corrected GPT-4's predictions into consensus labels and evidence-sentence sets. Their TrialGPT-Matching module reaches 87.3% criterion-level accuracy against an expert ceiling of 88.7–90.0%. We reuse their annotations but replace the frozen prompted GPT-4 with a *fine-tuned* small open model.

Open decoder-only LLMs. We build on Qwen3 [2], an Apache-2.0 family combining RMSNorm [3], rotary po-

sition embeddings [4], grouped-query attention (GQA) [5], per-head QK-normalisation, and a SwiGLU [6] MLP. Understanding these components is what makes a from-scratch forward pass feasible.

Parameter-efficient fine-tuning. LoRA [7] freezes the base weights and learns low-rank update matrices, cutting trainable parameters by orders of magnitude; it is what let us fine-tune a 4B model on a single 24 GB GPU.

Efficient attention and decoding. Online-softmax normalisation [8] computes the softmax in a single streaming pass with a running maximum and denominator; FlashAttention [9] turns this into a tiled, IO-aware kernel that never materialises the full attention matrix. Our Task-4 kernel is a didactic pure-PyTorch version of the same idea. Speculative decoding [10], [11] drafts several tokens with a cheap model and verifies them in one pass of the target, yielding identical output to greedy decoding whenever the draft is accepted. Earlier eligibility work relied on NLI-style encoder models [12]; we instead use a generative decoder so that the same model can also write a natural-language explanation.

III. Dataset and Task Formulation

We use `ncbi/TrialGPT-Criterion-Annotations` [1]. Each row pairs a patient note (a numbered list of sentences) with one trial criterion (inclusion or exclusion) and carries an expert consensus label and the set of evidence sentence indices. We drop the auxiliary `gpt4_*` columns from training and prediction, keeping them only to compute the GPT-4-vs-expert ceiling referenced below.

Labels. Five classes, following the dataset: for *inclusion* criteria {`included`, `not included`, `not enough information`, `not applicable`}; for *exclusion* criteria {`excluded`→`not excluded`, `not enough information`, `not applicable`}. The model must therefore condition its label space on the criterion type.

Target. We ask the model for strict JSON, {`"label": <class>, "evidence": [<sentence ids>]`}, which makes both outputs machine-checkable and lets an invalid parse count as its own wrong class.

Split. To prevent a patient's sentences from appearing in both train and test, we split *by patient*: the unique patient IDs are sorted and shuffled with a fixed seed (NumPy default RNG, seed 42) and partitioned 37/8/8 patients, giving 603 training, 206 validation, and 206 test rows. The longest prompt is 574 tokens, so nothing is truncated.

IV. Method

A. LoRA Fine-Tuning

We attach LoRA adapters (rank 16, α=32, dropout 0.05) to every attention and MLP projection, and train for 3 epochs with AdamW, learning rate 2×10^{-4}, a cosine schedule, and 5% warmup. The prompt presents the patient note first, then the criterion and the required JSON schema; the cross-entropy loss is *masked to the answer tokens only*, so the model is graded on the JSON it must produce, not on reciting the note. Gradient checkpointing plus `enable_input_require_grads()` and a batch size of 1 with 16-step accumulation keep the 4B model inside 24 GB.

B. From-Scratch Decoder and KV Cache (Task 3)

We reimplement the Qwen3 forward pass in PyTorch, loading the published HF weights directly: RMSNorm, per-head QK-Norm, RoPE applied at explicit token positions, GQA (32 query heads sharing 8 KV heads, head dimension 128), the SwiGLU MLP, and the tied LM head. A boolean flag folds the LoRA deltas into the linear layers so the *same* module serves the base and fine-tuned models.

The KV cache preallocates, per layer, a tensor of shape [batch, kv_heads, max_len, head_dim] and a length pointer. Prefill fills positions $0{:}T$; each decode step writes one column at the pointer and attends over the filled prefix, turning the per-step cost from $O(T^2)$ recomputation into $O(T)$. A `crop()` method rewinds the pointer, which is what makes speculative rollback (Section IV-D) cheap and exact.

C. Tiled Online-Softmax Attention (Task 4)

Rather than form the full $T_q \times T_k$ score matrix, the kernel walks key/value blocks while maintaining, per query, a running maximum m and denominator ℓ and rescaling the partial output when a new block raises the maximum—the online-softmax recurrence [8] underlying FlashAttention [9]. Causality uses a bottom-right-aligned mask so that a cached query at absolute position p attends to keys $0{:}p$ even when $T_q \neq T_k$. We keep it in readable PyTorch: the point is to prove we can reconstruct the kernel's numerics, not to beat a fused kernel.

D. Greedy Speculative Decoding (Task 5)

A Qwen3-0.6B draft (identical tokenizer) proposes k tokens greedily; the 4B target scores all $k{+}1$ positions in one forward pass; we accept the longest prefix where target and draft agree, emit the first target-corrected token, and `crop()` both caches back. Because acceptance is defined by exact greedy agreement, the emitted sequence is *identical* to plain greedy decoding—so this is a pure latency optimisation, and we report it as such.

E. Label-then-Explain Pipeline

For the app we use a two-call design. The fine-tuned model (LoRA on) emits the label and evidence IDs; the *same* base model (LoRA off) then writes a one-paragraph explanation constrained to the cited sentences only, with a deterministic template fallback when no evidence is cited. This keeps the explanation grounded in text the model actually pointed at, rather than free association.

F. Browser App (Task 6)

A Gradio app has two tabs. *Screen* runs every criterion of a trial against a patient note, shows each label with its highlighted evidence and explanation, rolls them into a rule-based overall verdict, and exports a downloadable HTML report with provenance. *Inference lab* exposes the systems work directly: KV-cache vs. recompute, the three attention variants, and speculative vs. greedy, each reporting tokens/s so a viewer can reproduce the benchmarks live.

V. Experimental Setup

Training, evaluation, and benchmarks run on one NVIDIA RTX PRO 4000 Blackwell (24 GB, bf16) with PyTorch 2.11/CUDA 12.8 and Transformers 5.22. Decoding is greedy with a 64-token answer budget. We report label **accuracy**, **macro-F1** (the unweighted mean of per-class F1, so rare classes count as much as common ones), per-class precision/recall/F1, the confusion matrix, and the **invalid-JSON rate**. Evidence retrieval is scored as sentence-set micro- and macro-F1. Correctness of the hand-written decoder is checked by a 27-test CPU suite on a tiny random Qwen3 in float32, asserting tiled = reference = SDPA, ours = HF logits, cached = uncached = HF generate, speculative = target greedy, and the LoRA flag = PEFT; all 27 pass.

VI. Results and Discussion

A. Classification and Evidence

Table I gives the headline numbers. Fine-tuning helps both models enormously: the 4B model gains +0.27 accuracy and +0.17 evidence micro-F1, and the 1.7B model gains +0.50 accuracy. No fine-tuned run ever emits invalid JSON. Figure 1 shows that validation accuracy and evidence-F1 both peak around epoch 2; macro-F1 is noisier because the rare classes (Table II) swing it. We therefore evaluate from the best-validation checkpoint.

Two honest caveats. First, macro-F1 (0.675 for the 4B) sits well below accuracy (0.762) because the minority classes are hard (next subsection). Second, the 1.7B model reaches respectable *accuracy* (0.714) but poor *macro*-F1 (0.452): it wins on the common classes and collapses on the rare ones, which is exactly what macro-F1 is designed to expose.

Is 0.76 "bad"? No—it is near the practical ceiling. On this same test split, the dataset's own GPT-4 labels agree with the expert consensus at only 0.903 accuracy / 0.906 macro-F1, and that figure is *inflated* because the gold labels were produced by physicians *editing GPT-4's* outputs, so GPT-4 is being graded partly against itself. A 4B open model at 0.76, with zero invalid output, is a strong result in that light.

TABLE I
Test-set results (206 pairs, greedy). Base vs. LoRA fine-tuned.

Model	Variant	Acc	Macro-F1	Ev. micro-F1	Ev. m
Qwen3-4B	base	0.495	0.554	0.444	0.
	tuned	**0.762**	**0.675**	**0.617**	**0.**
Qwen3-1.7B	base	0.218	0.259	0.262	0.
	tuned	0.714	0.452	0.542	0.
GPT-4 label vs. expert (ceiling)		0.903	0.906	–	

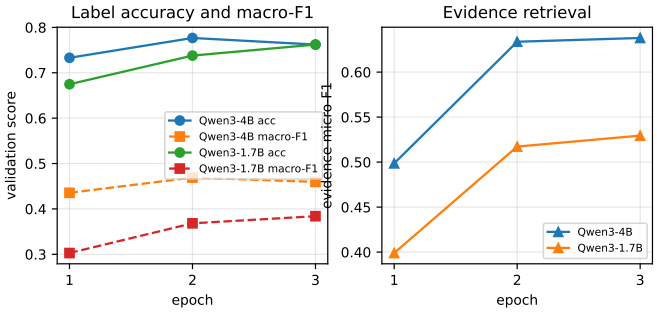


Fig. 1. Validation label accuracy / macro-F1 (left) and evidence micro-F1 (right) over three epochs. Both models peak near epoch 2.

B. Per-Class Behaviour

Table II and the confusion matrix in Figure 2(b) explain where the macro-F1 goes. The 4B model is precise on the positive classes—`included` precision 0.947, `not included` precision 1.000—but its *recall* on the minority classes is low (`not included` 0.286, `not applicable` 0.533): it is conservative and prefers the safe "cannot tell" answer. The majority class `not excluded` is both common and easy (F1 0.850). Split by criterion type, inclusion criteria are handled better (accuracy 0.787, macro-F1 0.712) than exclusion criteria (0.748 / 0.560), because exclusion reasoning more often hinges on an absent fact. Compared with the base model (Figure 2(a)), fine-tuning mostly sharpens the `not excluded` and `not enough information` columns.

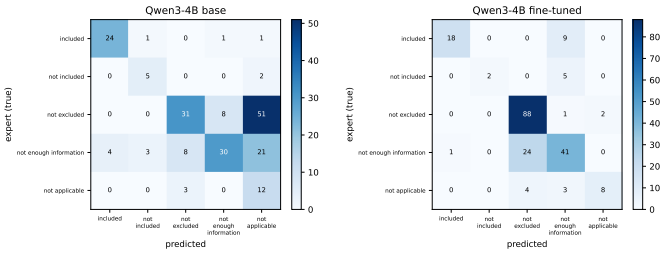
C. Error Analysis

The 4B model makes 49 label errors on the 206 test pairs. Table III groups every one of them. Three failure modes dominate and are worth naming:

1) **Over-confident "not excluded" (24 cases, 49% of errors).** When an exclusion criterion is genuinely unsupported by the note, the model says `not excluded` instead of `not enough information`. Example: *"Patients with previous axillary surgery"* with no mention of such surgery—the model treats silence as absence rather than as missing information.
2) **Over-cautious on positives (9 + 5 cases).** `included` and `not included` criteria are demoted to `not enough information`. Example: *"diagnosed with Shock, ARDS, or both"* where the note does

TABLE II
PER-CLASS METRICS, QWEN3-4B FINE-TUNED, TEST SET.

Class	Precision	Recall	F1	n
included	0.947	0.667	0.783	27
not included	1.000	0.286	0.444	7
not excluded	0.759	0.967	0.850	91
not enough information	0.695	0.621	0.656	66
not applicable	0.800	0.533	0.640	15



(a) base (b) fine-tuned

Fig. 2. Confusion matrices for Qwen3-4B before and after fine-tuning (test set). Fine-tuning concentrates mass on the diagonal, but a `not enough information`→`not excluded` leak remains.

state ARDS (gold evidence [2,3]) but the model returns an empty evidence set and hedges.

3) `not applicable` **confusions (7 cases).** E.g. a birth-control criterion for a male patient is `not applicable`, but the model reads the sentence it correctly cited as merely insufficient.

Notably, class 1 and 2 are mirror images of the same miscalibrated threshold on "do I have enough evidence?"—evidence errors and label errors are correlated, and the single most valuable improvement would be better calibration of the `not enough information` boundary rather than more capacity.

D. KV Cache (Task 3)

Figure 3 and Table IV show the payoff. With a cache, decode throughput is roughly flat in prompt length (≈31 tokens/s) because each step attends over stored keys; without one, every step re-encodes the whole prefix, so throughput collapses from 20 tokens/s at a 128-token prompt to 2.6 at 1024. The speed-up grows with context: 1.5×, 4.8×, and 12.3× at 128/512/1024-token prompts (128 new tokens). Peak memory rises only marginally (7.86→8.40 GB), the cost of storing the keys and values.

E. Tiled Attention (Task 4)

Numerically, the tiled kernel is correct: in float32 it matches the reference and SDPA bit-for-bit across the 27-test suite, and its standalone error versus a float32 ground truth (Figure 4, right) stays at the same $\sim 10^{-3}$ rounding floor as the reference itself. On *speed* (Figure 4, left) the didactic Python kernel is slower than both the reference matmul and the fused SDPA (18 vs. 33 vs. 38 tokens/s in-model), which is expected—the win from tiling is memory traffic on a fused CUDA kernel, not Python-level blocking. We also observed that in bf16 the tiled and SDPA paths can diverge from the naive reference by one token over a long generation; this is a rounding effect on near-tied logits, not a logic bug (Section VI-G).

TABLE III
ALL 49 LABEL ERRORS (QWEN3-4B TUNED), BY GOLD→PREDICTED PAIR.

Gold label	Predicted label	Count
not enough information	not excluded	24
included	not enough information	9
not included	not enough information	5
not applicable	not excluded	4
not applicable	not enough information	3
not excluded	not applicable	2
not excluded	not enough information	1
not enough information	included	1

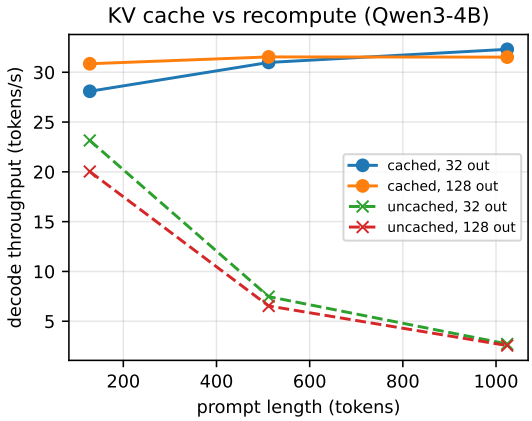


Fig. 3. Decode throughput with and without the KV cache (Qwen3-4B). Cached decoding is near-constant; recomputation degrades sharply with prompt length.

F. Speculative Decoding (Task 5)

Figure 5 is a negative but instructive result. Speculative decoding produces token-identical output to greedy, yet every configuration is a *slowdown* (speed-up < 1). The cause is acceptance: the 0.6B draft agrees with the 4B target only ∼52% of the time at k=2, falling to ∼22% at k=8, so most of the draft's k tokens are thrown away and the extra draft forward passes are not amortised. The closest to break-even is short, templated JSON (k=4: mean speed-up 0.86, acceptance 0.56, best single run 0.98), where the draft can often guess the rigid {`"label": ...`} scaffold. The lesson matches the literature [10]: speculative decoding pays off only when the draft is both much cheaper *and* frequently right, which a 0.6B/4B pair on free-form clinical text is not.

G. Numerical Equivalence

In float32 our cache, tiled kernel, and speculative path match Hugging Face exactly (27/27 tests). In bf16 on

TABLE IV
DECODE THROUGHPUT (TOKENS/S), 128 NEW TOKENS.

Prompt tokens	Cached	Uncached	Speed-up
128	30.9	20.0	1.5×
512	31.5	6.5	4.8×
1024	31.5	2.6	12.3×

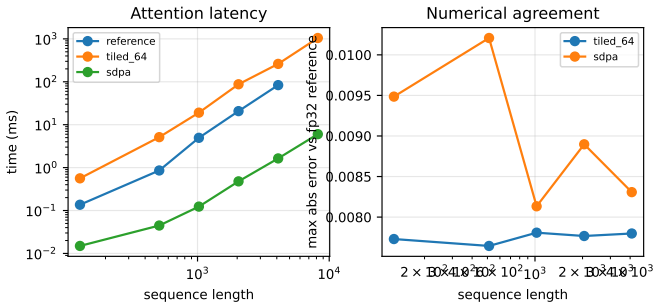


Fig. 4. Attention variants. Left: latency vs. sequence length (log–log). Right: max absolute error vs. a float32 reference—tiled tracks the reference's own rounding floor.

GPU, the cached decoder matches its own uncached path exactly in some configurations but can diverge by one token in others; the last-step logit gap at every such divergence is ≤ 0.31, i.e. a near-tie resolved differently by accumulated rounding. We report this openly rather than hide it: it is a property of low-precision accumulation, confirmed by the clean float32 equivalence, not an implementation error.

VII. Limitations

The test set is small (206 pairs, 8 patients), so per-class numbers on the rarest classes (e.g. `not included`, n=7) have wide error bars. The patient-disjoint split controls patient leakage but not trial-template leakage. Our tiled kernel is didactic and not speed-competitive with fused kernels. The speculative draft/target pair was not tuned (a distilled or domain-adapted draft would raise acceptance). Finally, we evaluate single-criterion decisions; rolling them into a trial-level recommendation (as the app does) adds aggregation rules we did not formally validate.

VIII. Conclusion

A 4B open decoder, LoRA-fine-tuned on expert criterion annotations, reaches 0.76 label accuracy and 0.62 evidence-F1 with zero malformed outputs—close to the inflated GPT-4 ceiling—while a hand-built inference stack makes its decoding transparent: a KV cache worth up to 12×, a provably-correct tiled attention kernel, and an honestly-measured speculative decoder. The dominant remaining error is a miscalibrated "not enough information" boundary, which points at calibration rather than scale as the next lever.

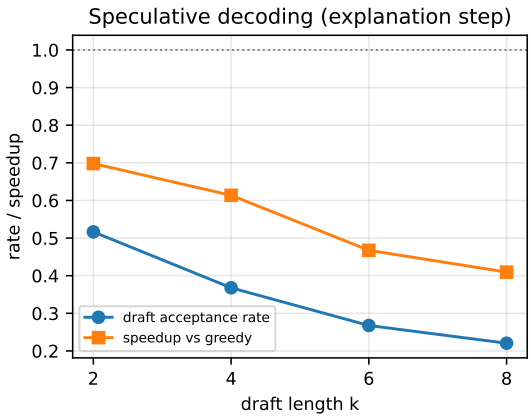


Fig. 5. Speculative decoding on the explanation step. Acceptance and speed-up both fall as the draft length k grows; speed-up stays below 1.

Appendix A
Declaration of AI Assistance

This project was carried out with an AI coding assistant (Claude, via Claude Code). The assistant contributed to: scaffolding the training/evaluation/benchmark pipeline and the from-scratch decoder, debugging GPU out-of-memory and precision issues, orchestrating the training runs, generating the figures in this report, and drafting its prose. All design decisions, the experimental setup, the choice of model and split, and the final claims were directed and verified by the author. All reported numbers were produced by the committed code on the stated hardware and were checked against the raw result files; no results were fabricated or estimated. The 27-test equivalence suite was written to independently verify the hand-built decoder rather than to assert its correctness by inspection.

References

[1] Q. Jin, Z. Wang, C. S. Floudas, F. Chen, C. Gong, D. Bracken-Clarke, E. Xue, Y. Yang, J. Sun, and Z. Lu, "Matching patients to clinical trials with large language models," *Nature Communications*, vol. 15, no. 1, p. 9074, 2024.

[2] A. Yang *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[3] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[4] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[5] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[6] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[8] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[10] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML)*, 2023.

[11] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.